KRUSKAL'S ALGORITHM

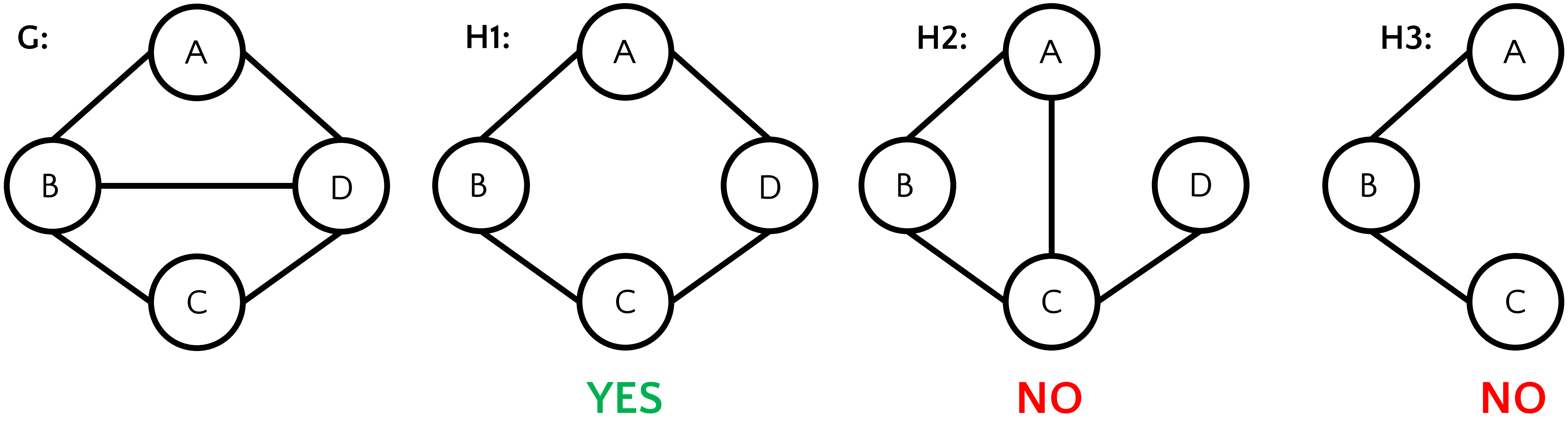
PREPARED BY
SWAPNIL BISWAS

GREEDY ALGORITHM

- Always takes the best at each phase
  - Without regards for future consequences
- Hoping that by choosing a local optimum at each step, it will end up at a global optimum
- If there are multiple options at any phase that indicates multiple global optimum
- Can be applied if a problem has optimal substructure property
- Can not be applied if a problem has overlapping sub problem property
- *Calculating Minimum Spanning Tree of a graph is solved by Greedy approach*

SPANNING SUB GRAPH

- H is a spanning sub graph of G if
  - V(H) = V(G)
  - E(H) ⊆ E(G)



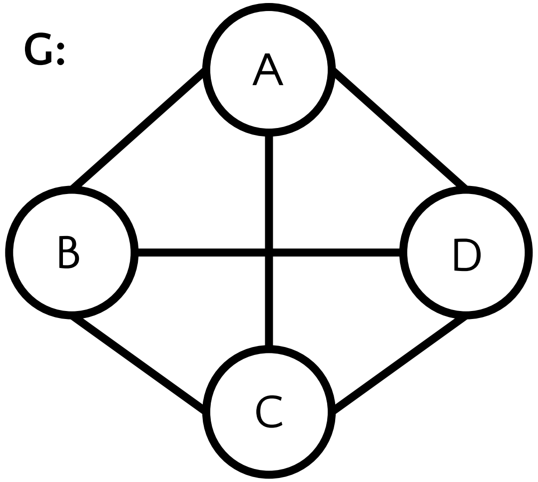
H1 is a spanning sub graph of G but H2 and H3 is not

SPANNING TREE

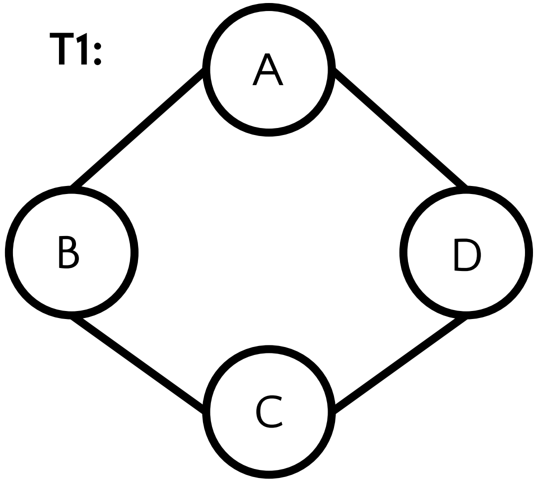
❑ T is a spanning sub tree of G if

- $V(T) = V(G)$
- $E(T) \subseteq E(G)$
- T is a tree

G:

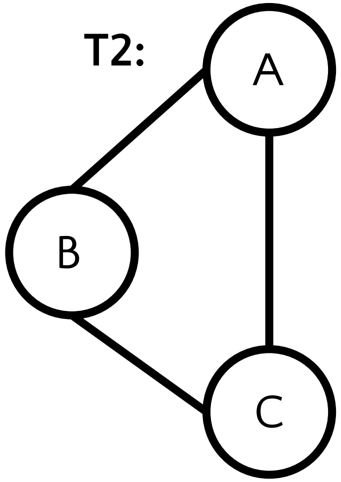


T1:



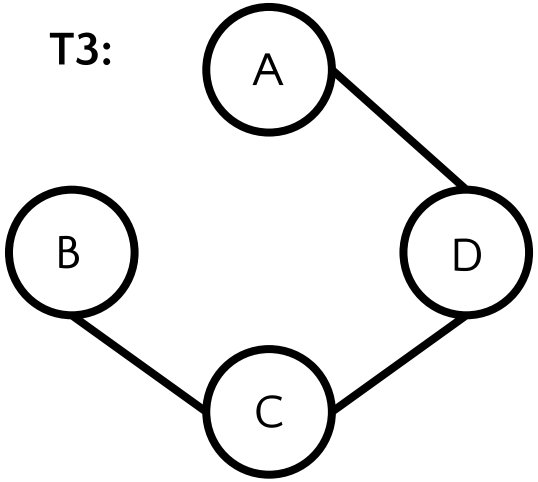
NO

T2:



NO

T3:

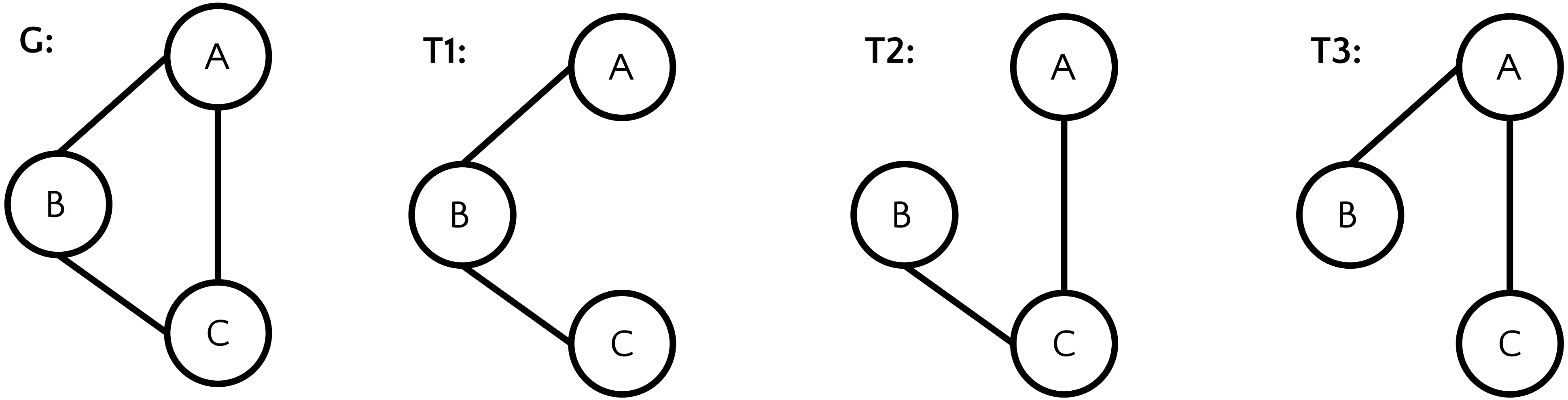


YES

T3 is the only Spanning Tree of G

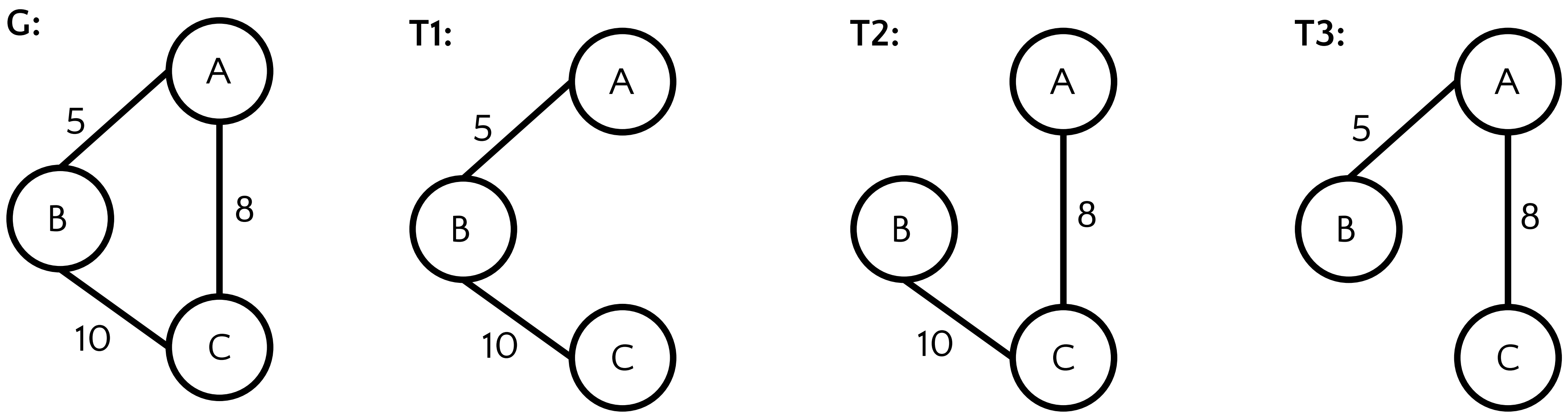
MULTIPLE SPANNING TREE

- A graph can have multiple spanning trees



WEIGHTED SPANNING TREE

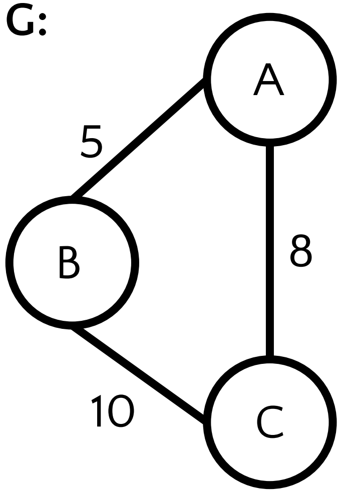
- Edges are weighted



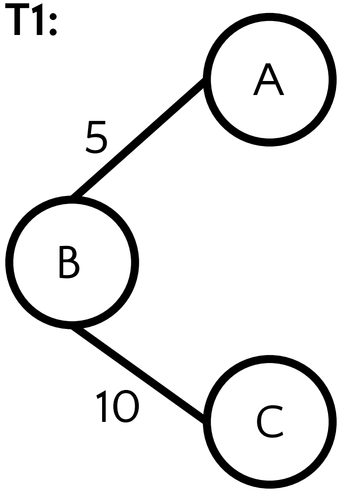
MINIMUM SPANNING TREE (MST)

- The spanning tree of G having minimum weight sum

G:

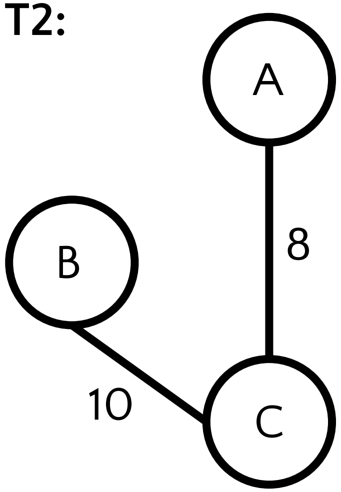


T1:



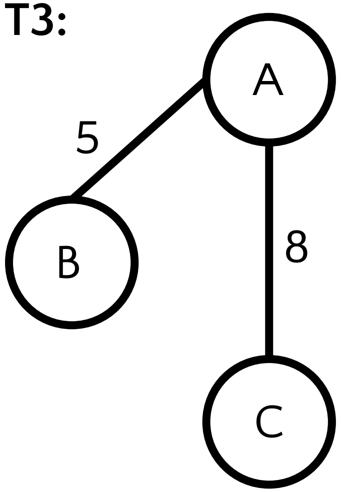
Total Weight: 15

T2:



Total Weight: 18

T3:



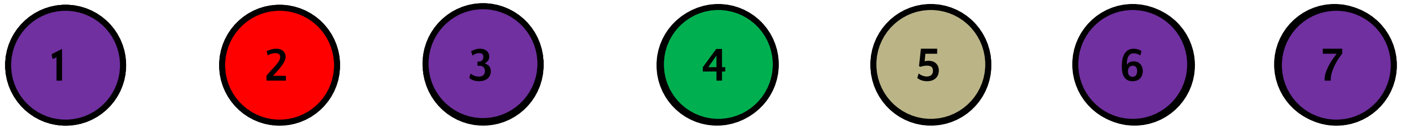
Total Weight: 13

<u>T3 is the Minimum Spanning Tree of G</u>

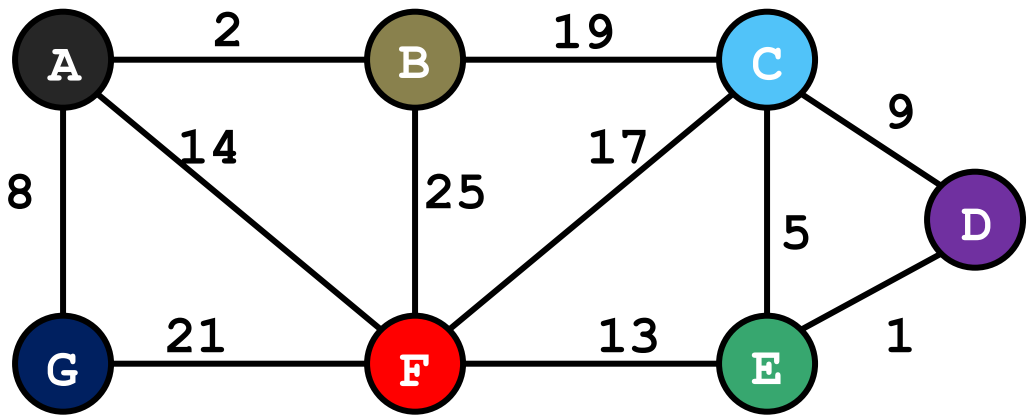
DISJOINT SET OPERATIONS

- makeSet() O(1)
- union(a,b) O(1)
- findSet(a) O(logn)

- union(1,3)
- union(6,7)
- union(6,1)



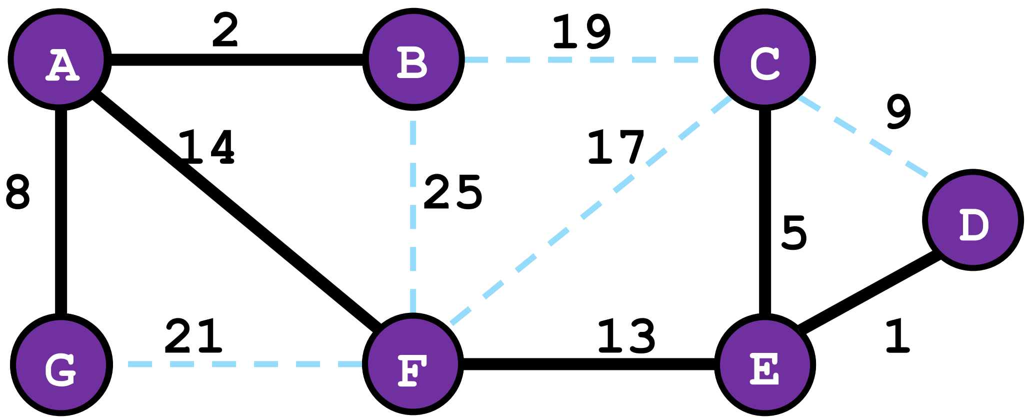
KRUSKAL's ALGORITHM (SIMULATION)



- Sort the edges in ascending order
- Create a set T for the MST with no edges

E	1	D
A	2	B
C	5	E
A	8	G
C	9	D
E	13	F
A	14	F
C	17	F
B	19	C
G	21	F
B	25	F

KRUSKAL's ALGORITHM (SIMULATION)



- Sort the edges in ascending order
- Create a set T for the MST with no edges
- Connect n-1 edges with no cycle
- Total cost: **43**

E	1	D	✓	
A	2	B	✓	
C	5	E	✓	
A	8	G	✓	
C	9	D	X	
E	13	F	✓	
A	14	F	✓	DONE!
C	17	F		
B	19	C		
G	21	F		
B	25	F		

KRUSKAL's ALGORITHM

```
Kruskal()
{
   T = ∅;
   for each v ∈ V
      MakeSet(v);
   sort E by increasing edge weight w
   for each (u,v) ∈ E (in sorted order)
      if FindSet(u) ≠ FindSet(v)
         T = T U {{u,v}};
         Union(FindSet(u), FindSet(v));
}
```

CORRECTNESS OF KRUSKAL's ALGORITHM

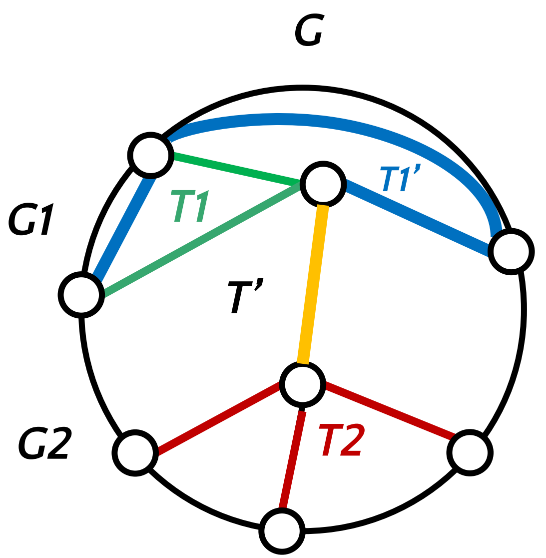
- Let T is the tree generated by Kruskal's algorithm
- We assume T* is the MST
- As Kruskal's algorithm checks cycle at every iteration so the solution contains no cycle
- As Kruskal's algorithm operates on a connected graph and completes iteration after adding n-1 edges so the solution is a tree

- Let T: $\underbrace{e_1\ e_2\ \dots\dots\dots\ e_i}_{T_1}$ | x x x
- Let T*: $\underbrace{e_1\ e_2\ \dots\dots\dots\ e_i^*}_{T_1^*}$ | x x x

- e_i and e_i^* are the first edges miss matched in T and T*
- So, $w(e_i) <= w(e_i^*)$
- So, $w(T_1^*) >= w(T_1^*) - w(e_i^*) + w(e_i)$
- So, $w(T_1^*) >= w(T1)$
- Prove recursively

OPTIMAL SUBSTRUCTURE PROPERTY OF MST

- ***Each Sub tree in a Minimum Spanning Tree is a Minimum Spanning Tree***



- Let *T* is the minimum spanning tree of *G*
- Let e = (u,v) is an edge that belongs to T
- Now we divide G into 2 sub graph G1 and G2
- Such that, u ∈ G1 and v ∈ G2
- We denote the spanning sub tree of T in G1 as T1 and G2 as T2
- We claim that if T is the MST of G then T1 is the MST of G1 and T2 is the MST of G2
- Now, w(T) = w(T1) + w(e) + w(T2) ………(i)
- Now for contradiction we assume, T1 is not the MST of G1
- Let, T1' (blue edges) is the MST of G1
- It implies that, w(T1') < w(T1)
- Now we find a new tree T' in G by connecting T1' and T2 by e
- So, we can write, w(T') = w(T1') + w(e) + w(T2)
- By replacing w(T1') by w(T1) we can write, w(T') < w(T1) + w(e) + w(T2) [As w(T1) > w(T1')]
- w(T') < w(T) that implies that T is not the MST of G
- So, if we claim T as the MST of G then T1' can't be the MST of G1, T1 will be the MST of G1
- We can prove it similarly for T2 as well as any sub tree of T

Thank You!